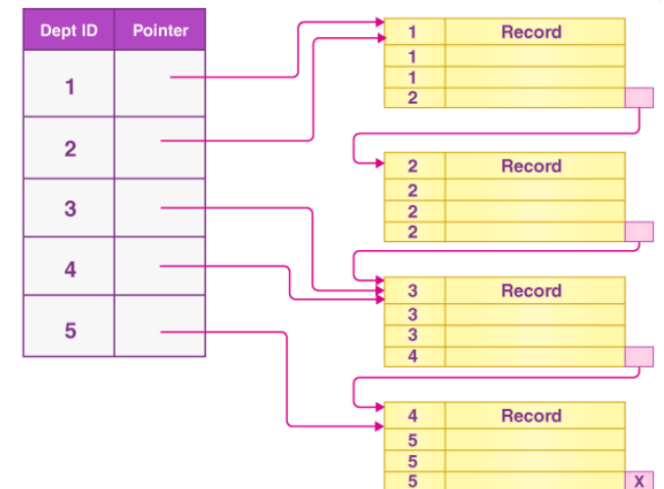
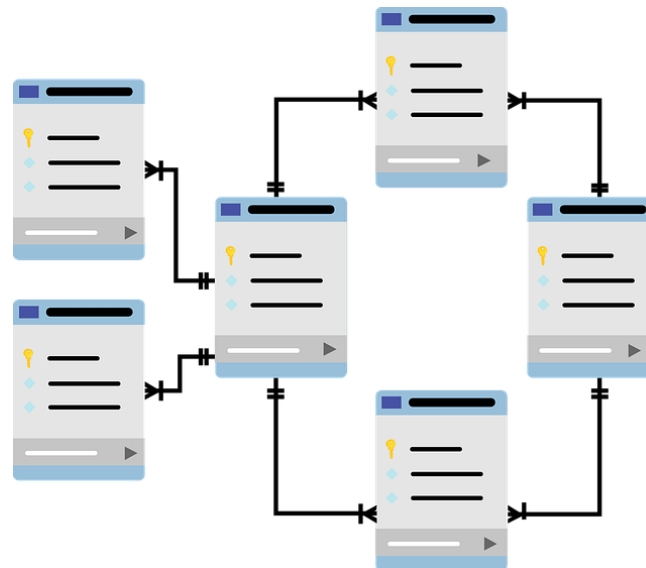
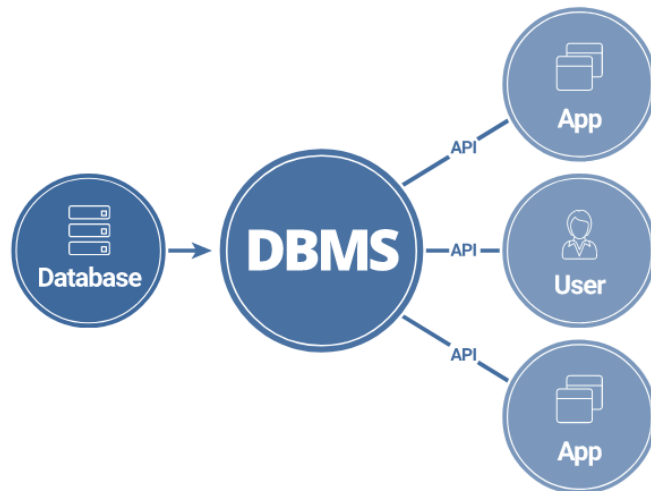


DBMS Concepts

By

(Aurora Computer Studies | auroracs.lk)



Introduction to DBMS

Overview

- Introduction to DBMS
- Database design
 - ER modeling
 - Normalization
- SQL

Data Management – Why Important?

- Keep data related to important entities like customers, products, assets, documents, etc.
- Record or keep track of transactions.
- Provide data/information for planning & decision making.
- Provide data for business analytics & data mining.

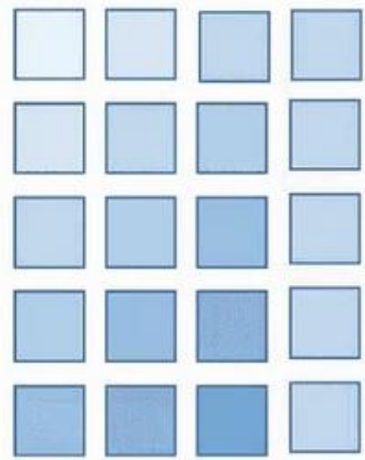


Tasks in Managing Data

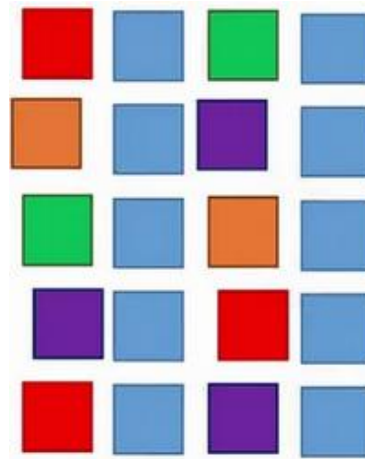
- Main tasks
 - Persist (save) data
 - Update & delete data
 - Retrieve data
- Other aspects
 - Safeguard data integrity
 - Backup & recovery
 - Secure data

Types of Data

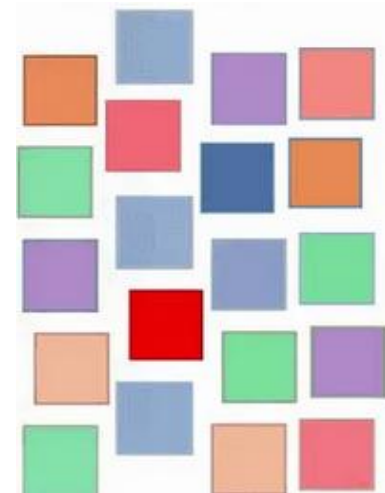
- Structured data
- Semi-structured data
- Unstructured data



Structured Data



Semi-Structured Data



Unstructured Data

Structured Data

- Data having well-defined logical/semantic structure.
- Best fit for;
 - Relational databases.
 - Easy to manipulate using SQL.
- Examples
 - Employee's information
 - Name, gender, contact details, etc.
 - Marks of students
 - Marks for each subject.
 - Hotel room reservation details
 - Name, contact number, from date, to date, etc.

id	name	age
1	Jim	28
2	Pam	26
3	Michael	42

id	subject	Teacher
1	Languages	John Jones
2	Track	Wally West
3	Swimming	Arthur Curry
4	Computers	Victor Stone

student_id	subject_id	grade
2	1	98
1	2	100
1	4	75
3	3	60
2	4	76
3	2	88

Unstructured Data

- Data having no or little structure.
 - Represent bulk of the generated data.
 - Also known as '**Big data**'.
 - Examples
 - Documents/ Texts
 - Web pages
 - E-mail messages
 - Photos/images
 - Audio
 - Videos (including CCTV outputs)
- 
- A decorative graphic in the bottom right corner consisting of several overlapping, colorful icons. These include a yellow folder, a red document, a blue image icon, a green video play button icon, and a red smartphone icon, all arranged in a cluster.

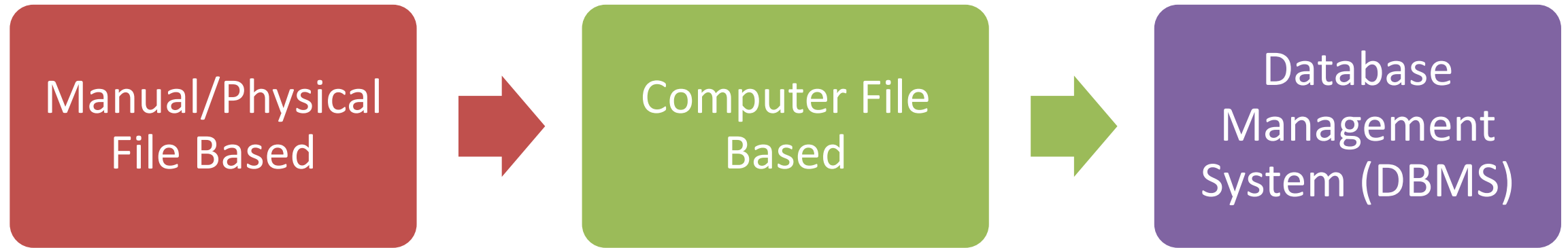


Semi-structured Data

- These are somewhat structured data, but does not fully conform to the formal structure of data models associated with relational databases.
- Examples
 - Web contents
 - Social media contents
 - XML/JSON based documents

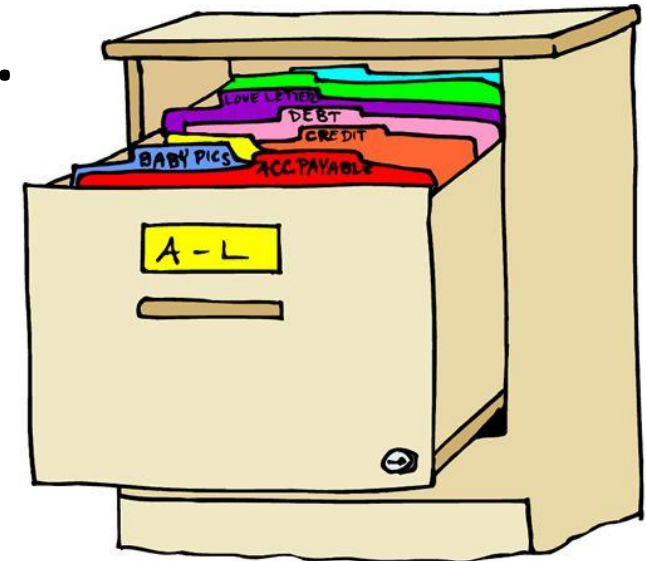


Evolution of Data Management

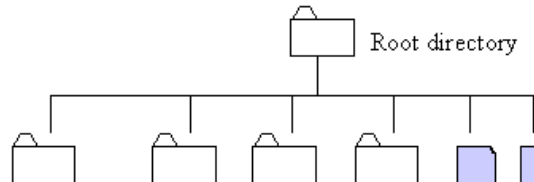


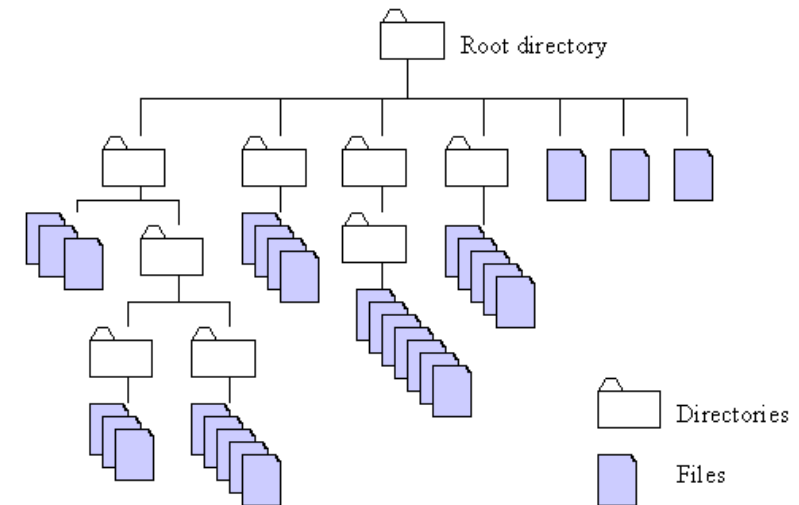
Manual/Physical File-based Systems

- Use of paper-based forms in files & folders.
- Use of manual indexes for faster retrieval.
- Example
 - Library books index card catalog (manual)
- Disadvantages
 - Natural degradation of physical media.
 - Inefficient to search/modify.
 - Difficult to transfer data.



Computer File-based Systems

- Use of computer files that are accessed and manipulated by computer programs.
 - Advantages
 - Need less storage than manual.
 - More efficient than manual.
 - Can transfer data electronically.
- 

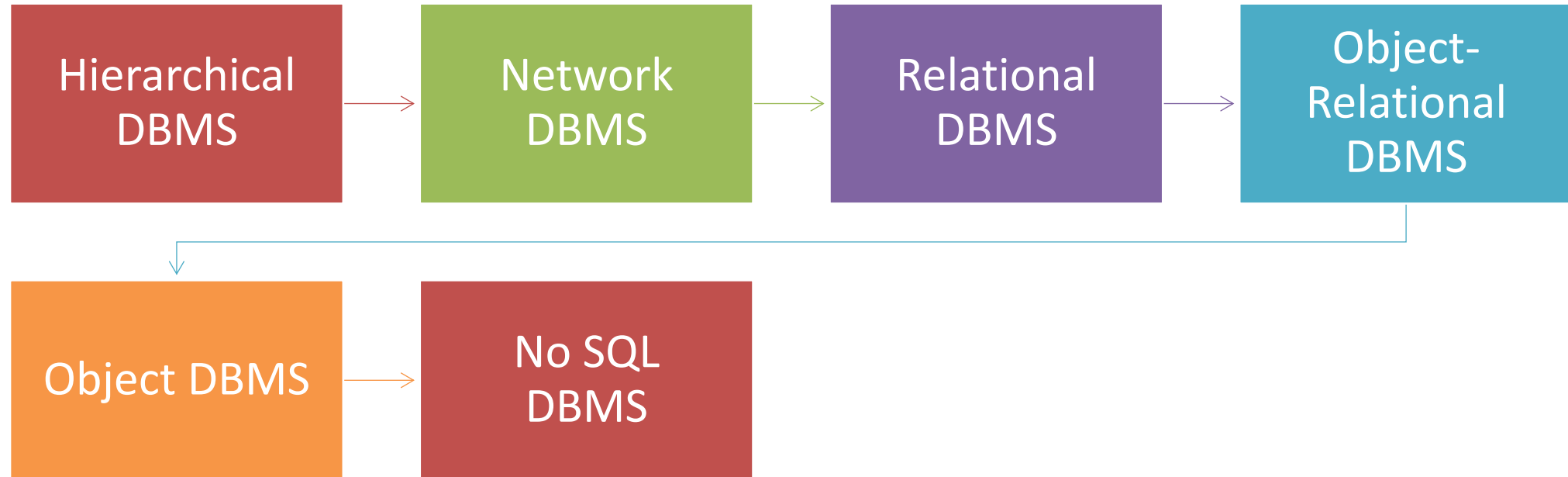


Database Management System (DBMS)

- Database is a centrally organized data repository.
- It provides higher level abstraction of data for applications.
 - Internally data is still saved in files.
- Also provide additional services such as;
 - Security
 - Integrity
 - Backup & recovery

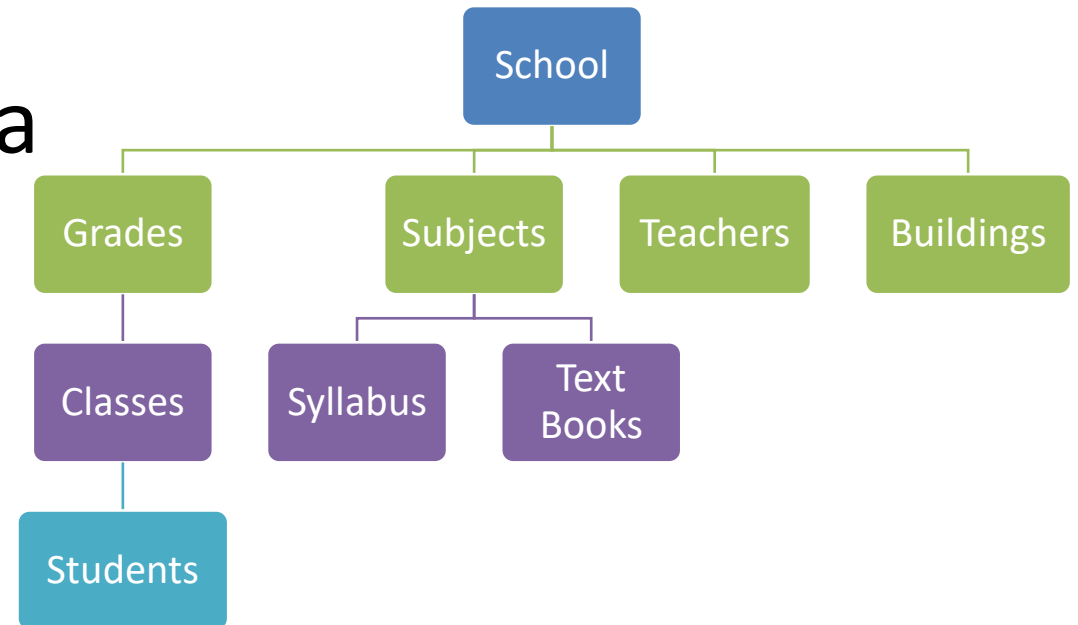


Evolution of DBMSs



Hierarchical DBMS

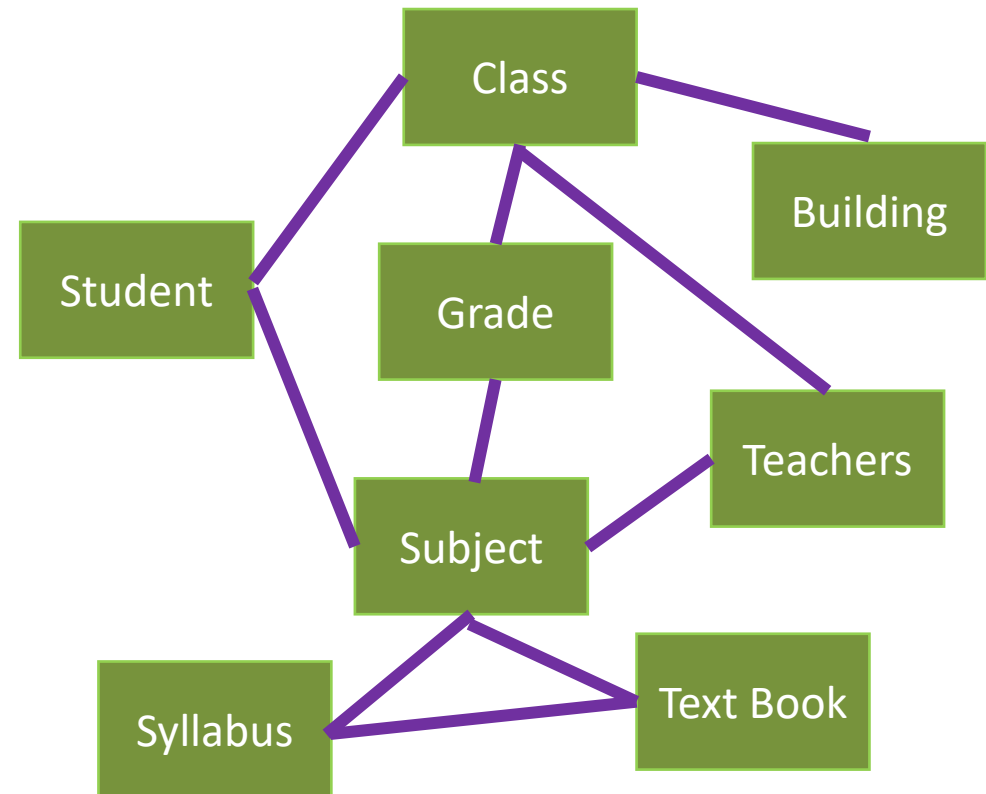
- A database is created as a hierarchy of tables.
- The idea was similar to hierarchy in traditional organizational structure.
- Advantages
 - Tight control over the data
- Disadvantages
 - Lack of flexibility



Network DBMS

- A database is created by freely interconnecting tables wherever related.
- Advantages
 - High flexibility
- Disadvantages
 - Very little control over the relationships

Note – Network DBMS has no connection with computer networks.



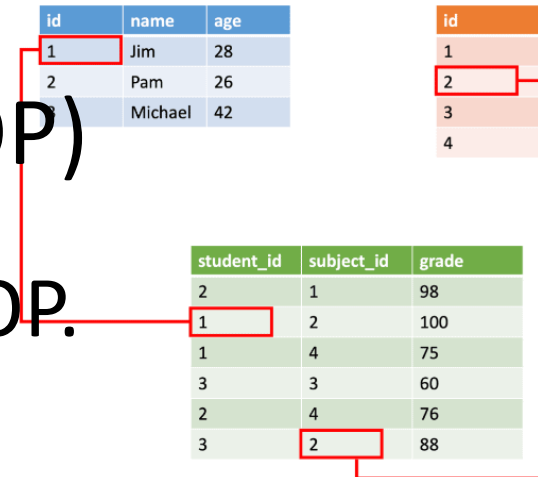
Relational DBMS

- A database designed based on relational theory.
 - Relations are made only by mapping a foreign key to the primary key of the related table.
- Advantages
 - Good combination of control and flexibility.
- Disadvantages
 - Not fully comply with modern Object-Oriented Programming (OOP) concepts.
 - Relational DBMSs evolved before OOP.

id	name	age
1	Jim	28
2	Pam	26
3	Michael	42

id	subject	Teacher
1	Languages	John Jones
2	Track	Wally West
3	Swimming	Arthur Curry
4	Computers	Victor Stone

student_id	subject_id	grade
2	1	98
1	2	100
1	4	75
3	3	60
2	4	76
3	2	88



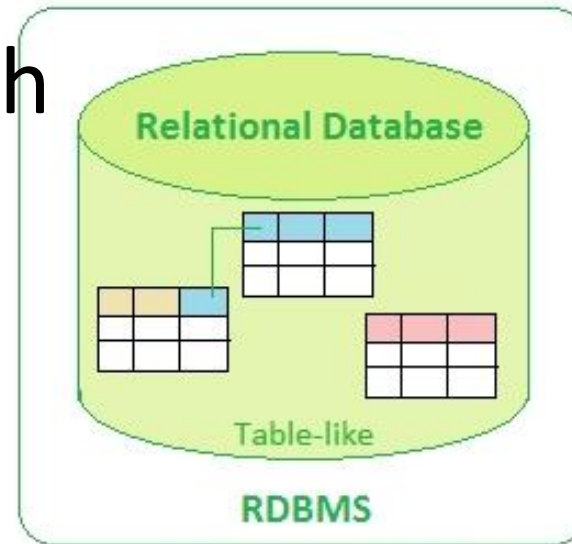
Relational DBMS - Examples

- Oracle DB
- Microsoft SQL DB
- MYSQL/Maria DB

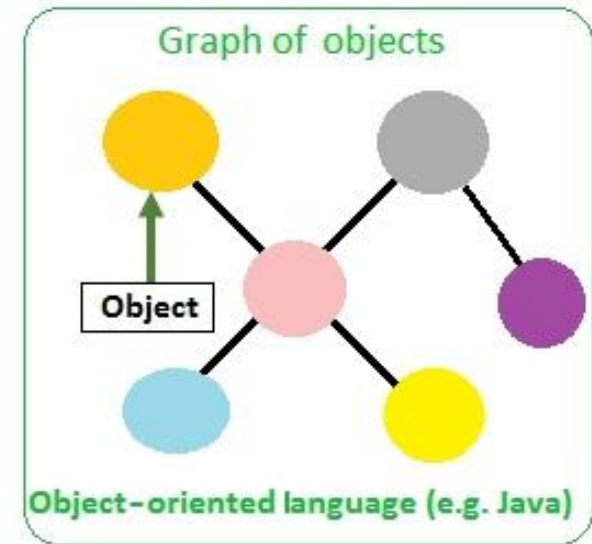


Object DBMS

- A DBMS that fully supports object-oriented concepts like encapsulation, inheritance and polymorphism.
- Advantages
 - Tight compatibility with OO programs.
- Disadvantages
 - Still an evolving technology.



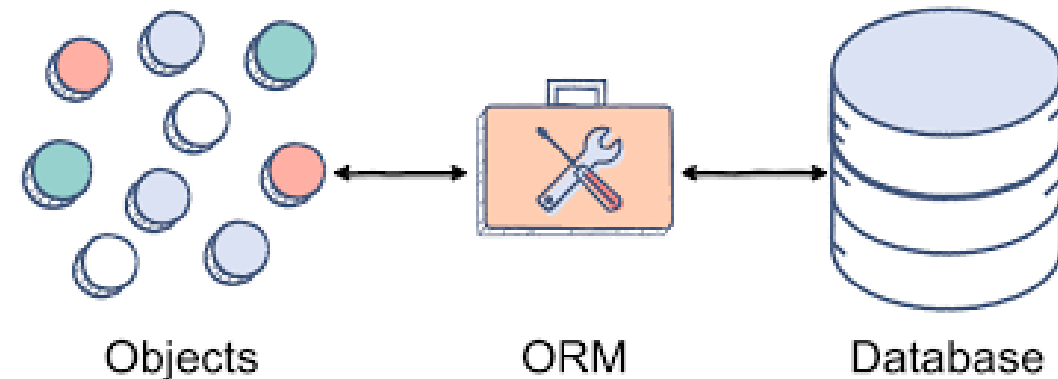
Relational Model



Object Model

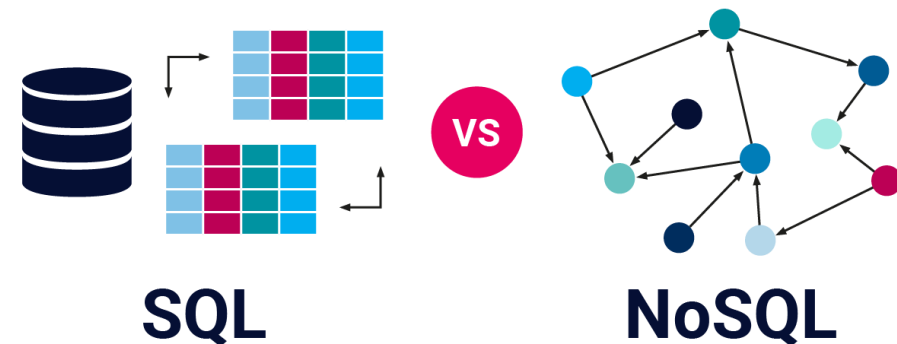
Object-Relational DBMS

- Object-relational database supports both relational concepts and some of the object-oriented concepts.
- Advantages
 - Support for both relational and OO concepts
- Disadvantages
 - Lack support for some OO concepts



NOSQL DBMS - Overview

- Use for handling unstructured & semi-structured data.
- It is more flexible for some applications like;
 - Social media
 - Content publishing
 - Storing session data
 - Mobile computing, etc..



ANSI/SPARC 3 Tier Architecture

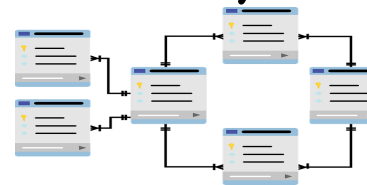
External/User schema

Forms + Reports + Views

A screenshot of a 'Customer Data Entry' form. It contains fields for Customer ID, Contact First Name, Contact Last Name, Billing Address, City, State (a dropdown menu), and Postal Code. A 'Record' button is at the bottom.

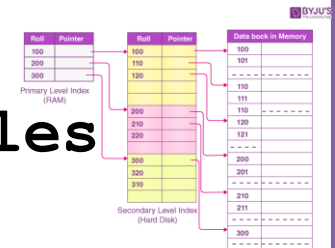
Logical/Conceptual
schema

Relations (Tables) + Relationships

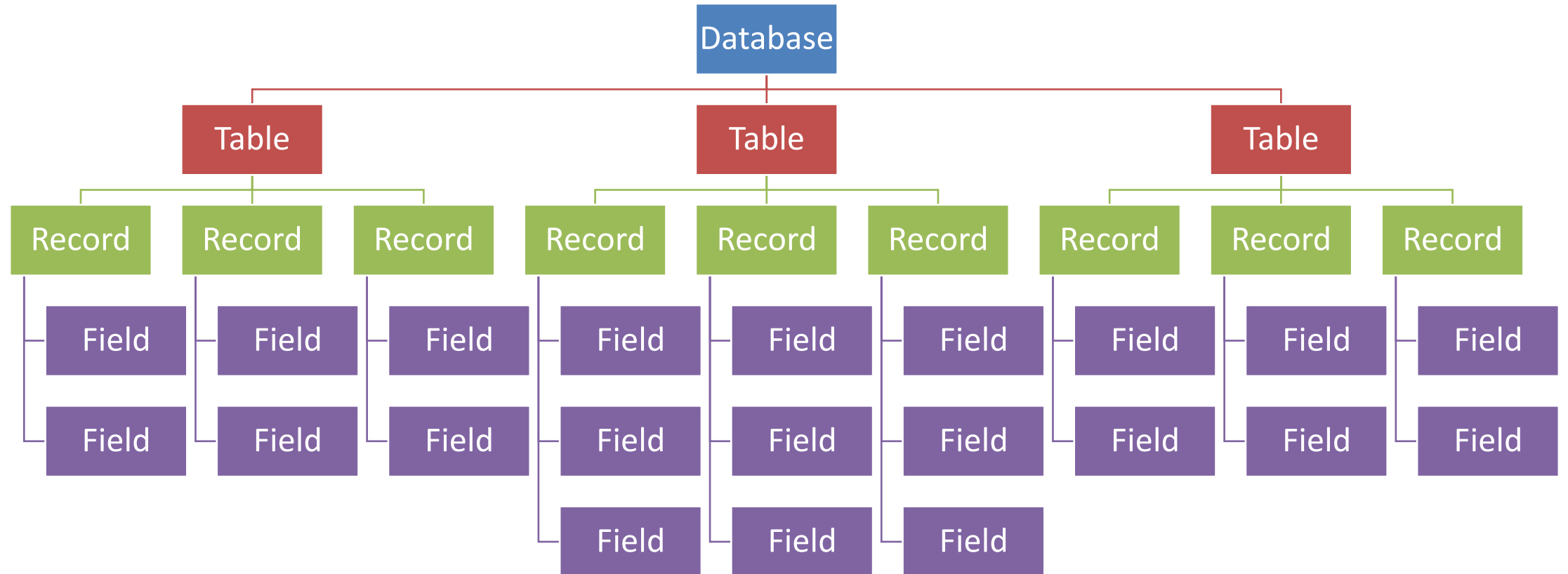


Internal/Physical
schema

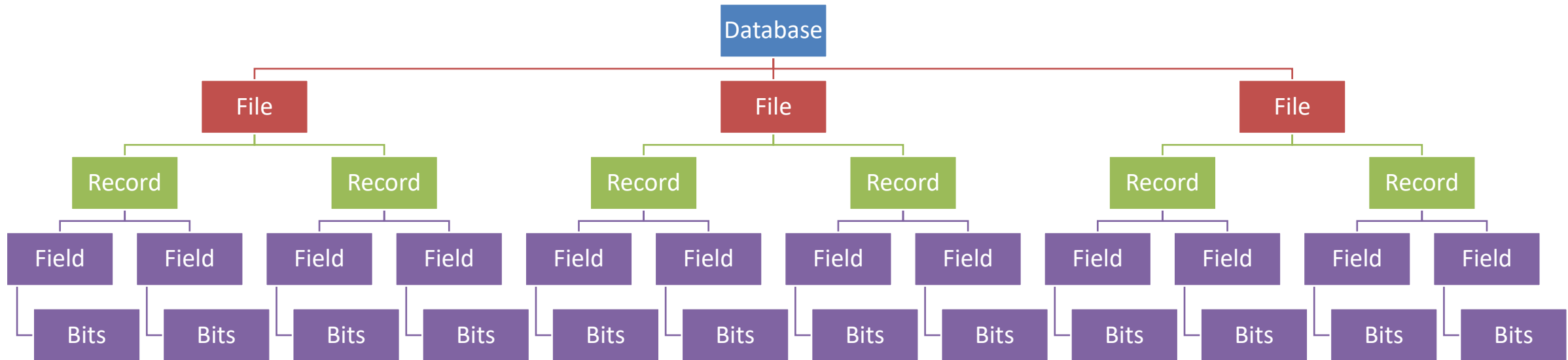
Data files + Index files



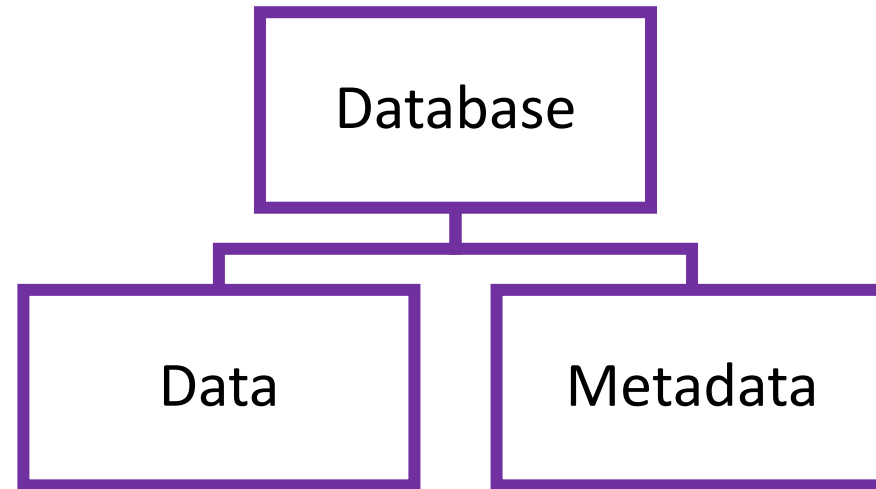
Logical Data Hierarchy



Physical Data Hierarchy



Data Vs Meta Data



Contains data records.
Also known as data view.

Resides in **data dictionary/system catalog**.
In some DBMS, we call this **design view**.

Examples

- The names of the tables
- Fields of each table
 - Data type, length, etc.
- Keys
- Integrity rules or constraints

Meta Data

- Meta data is data that describe the properties or characteristics of other data.
- Some of these properties include;
 - Name of the data item
 - Data type
 - Length
 - Minimum and maximum allowable values (where appropriate)
 - Rules or constraints
 - A brief description of data item

Database Designer

- Analyze/Understand data related requirements.
- Design the data base according to requirements.

Meta Data - Example

- Employee table

Name	Type	Length	Min	Max	Constraints	Description
EmpNo	Number	9			Unique	Employee No
Name	Character	30				Employee Name
Dept	Character	10				Dept. No.
Salary	Number	8	10000	100000		Employee Salary

Database Types Based on Usage & Roles

Types of DBMS by Usage

- **Personal DBMS**

- Use by single user.
- Use in smartphones and PCs.
- Examples
 - MS Access, Libre Office Base, etc..

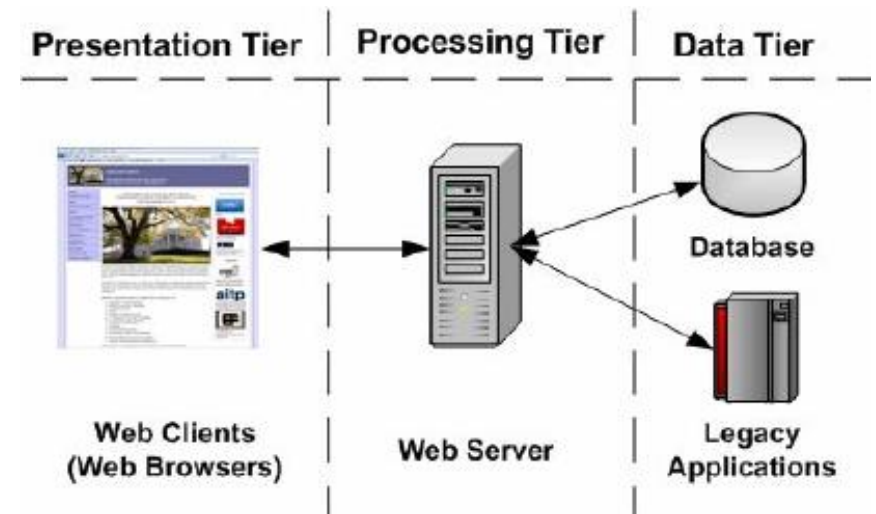
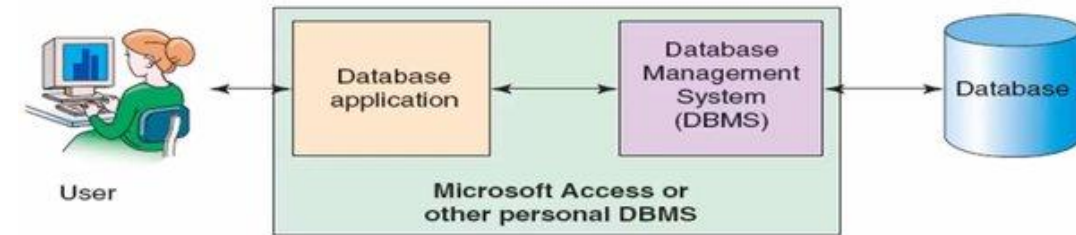
- **Workgroup DBMS**

- Use by team or a few users.

- **Enterprise DBMS**

- Use by business and other organizations.
- Key features
 - Support for transaction and concurrency handling.
 - High performance
- Examples
 - Oracle, MS SQL, MYSQL, etc..

Personal Database Systems



People Aspects

- Database administrator
- Database designer
- End-users

Database Administrator (DBA)

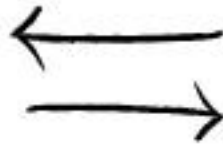
- Administering the DBMS and related software.
 - Acquiring software and hardware resources.
 - Install and setup data bases.
 - Authorizing access to the database.
 - Tuning the DBMS for best performance.

End User

- Use or get the benefit of a DBMS.
- Examples
 - Customer
 - Manager



End User



Customer Support



Developer

Database Design

Objectives

- Minimize data redundancy
- Enforce data integrity
- Other
 - Performance
 - Security

Terminology

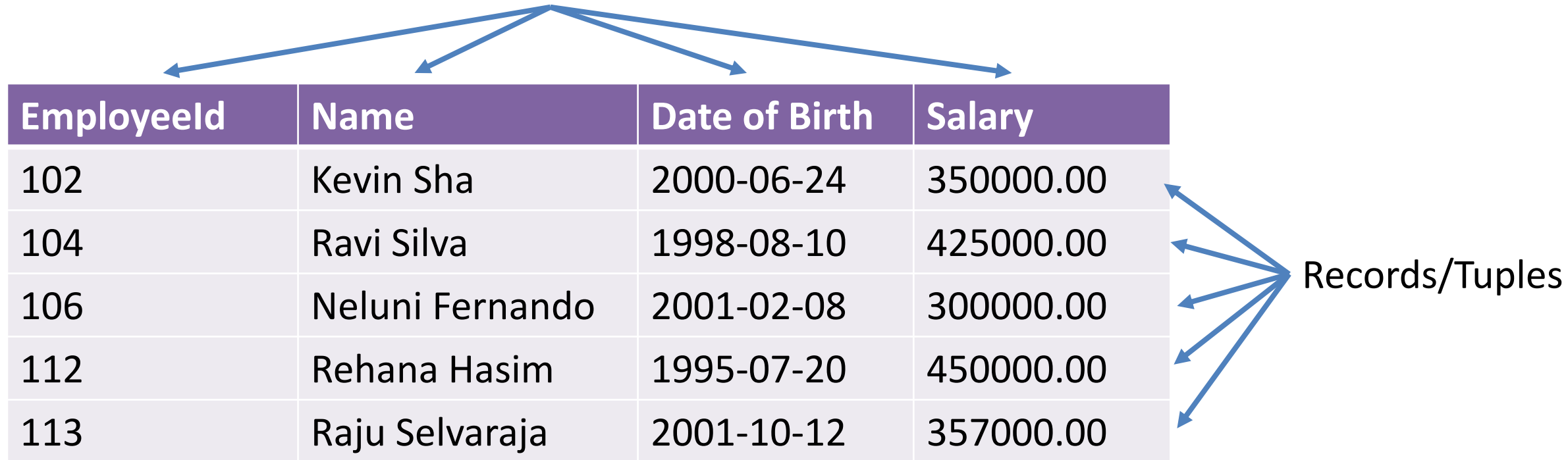
- Relation (Table)
 - Attribute/field
 - Record/tuple
 - Attribute domain of a field
 - Degree of a relation
 - Cardinality of a relation
 - Null value
 - Primary key
- Relationship
 - Degree of a relationship
 - Cardinality of a relationship
 - Foreign key

Relations

Relation (Table)

- A relation represent an entity or associative entity that we persist.

Fields/Attributes/Columns



EmployeeId	Name	Date of Birth	Salary
102	Kevin Sha	2000-06-24	350000.00
104	Ravi Silva	1998-08-10	425000.00
106	Neluni Fernando	2001-02-08	300000.00
112	Rehana Hasim	1995-07-20	450000.00
113	Raju Selvaraja	2001-10-12	357000.00

Records/Tuples

Table/Relation

Null Value

- For a record, the value for a field is not set or empty, we say that it's value is **null**.
- Example

Id	FirstName	LastName	DateOfBirth
1	Nimal	Perera	1980-10-10
2	Kamal	Fernando	null
3	Chamila	Gurusinghe	1989-09-10

Primary Key

- Selected field(s) (from candidate keys) to uniquely identify a record in the relation is called the primary key.
 - We must define only one primary key for a given relation.
- Example
 - We may select **StudentId** as the primary key from the two candidate keys of **StudentId** and **StudentNIC**.

Simple Vs. Composite Primary Key

- Simple primary key
 - A single field primary key
- Composite primary key
 - A multi-field primary key

Simple Primary Key

- If the selected primary key is a single field, we call it a simple primary key.
- Most of the domain keys and all surrogated keys are simple primary keys.
- Example
 - StudentId of student table
 - OrderId of order table

Composite Primary Key

- If the selected primary key contain more than one fields, we call it a composite primary key.
- The connecting table of a many-to-many relationships can have composite primary key (includes the two foreign keys for relating to the two tables).
- Example
 - {StudentId, CourseID} of connecting table for Student and Course tables having many-to-many relationship.

Domain Vs Surrogated Primary Key

- The selected primary key can be identified as one of the following types;
 - Domain primary key
 - Surrogated primary key

Domain Primary Key

- The selected primary key is visible and refer by (internal or external) users for domain or business purposes.
- Examples
 - Order number
 - Use by customers for enquiry.
 - Examination index number of a candidate
 - Use by internally & externally to identify a candidate.

Surrogated Primary Key

- The selected primary key is not visible and referred by (internal or external) users for domain or business purposes.
- It is usually a system auto generated unique number from a sequence.
- Use only as a primary key and also foreign key field in related tables.
- Examples
 - Order id
 - Use only internally by DBMS. There will be another order number for business references.
 - Account id
 - Use only internally by DBMS. There will be another account number for business reference.

Features of a Good Primary Key

- Uniquely identify an entity instance.
 - No possibility of duplicate value.
- Must not change from time to time.
- Preferably, not rely on the end user for uniqueness and accuracy.
 - Surrogated primary key is preferred.

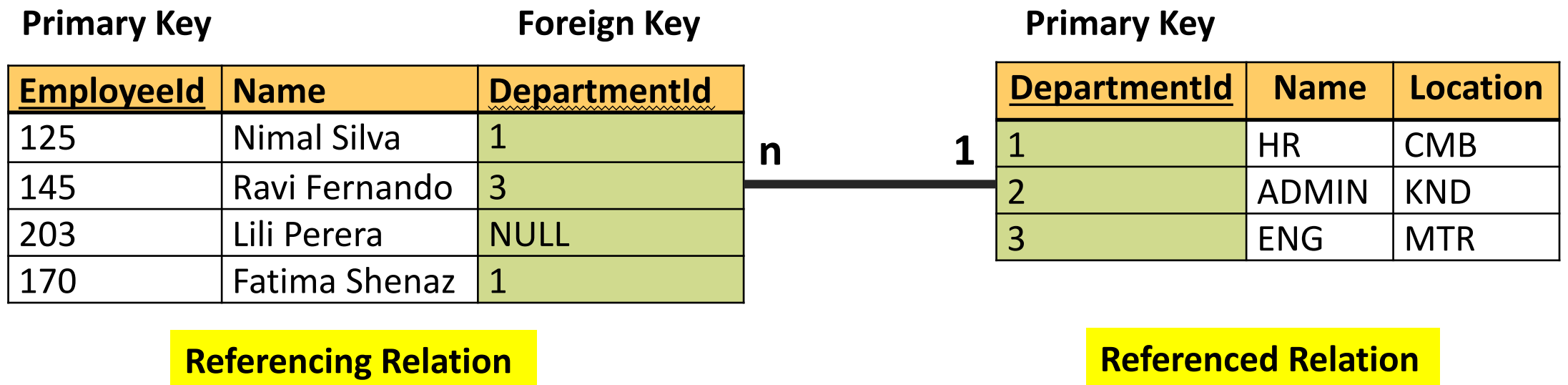
Relationships

Relationship – What?

- Relationship is a logical connection between entity instances (record).
- Examples
 - An employee works for a department
 - A student follows courses
 - Customer place an order
 - Payment is done for a order

Relationship – How ?

- A relationship is created using the concept of foreign key.
- Example



Foreign Key – What?

- A foreign key is a column in a table that is mapped to the primary key of the related table thereby creating a relationship.

Note

Since primary keys may be composite keys, the foreign keys may also be composite foreign keys. (Single primary/foreign keys are preferred in designing.)

Foreign Key – How?

- Create a field in the related table and map it to the primary key of the original table.
 - The data type of the foreign key field must be compatible with the data type of the primary key of the other table.
 - The name of the field of the foreign key may be different from the name of the primary key field of the other table.

Multiplicity (Cardinality) of a Relationship

- Multiplicity of a relationship determines how many entities involve in the relationship.
- Possibilities;
 - One-to-one relationship
 - One-to-many relationship
 - Many-to-many relationship

One-to-Many Relationship - What?

- If one instance of A is related to many instances of B, and one instance of B is related to one instance of A, then it is said to be a one-to-many relationship.

<u>department_id</u>	department_name
1	Human Resources
2	Engineering
3	Marketing

Department

1

n

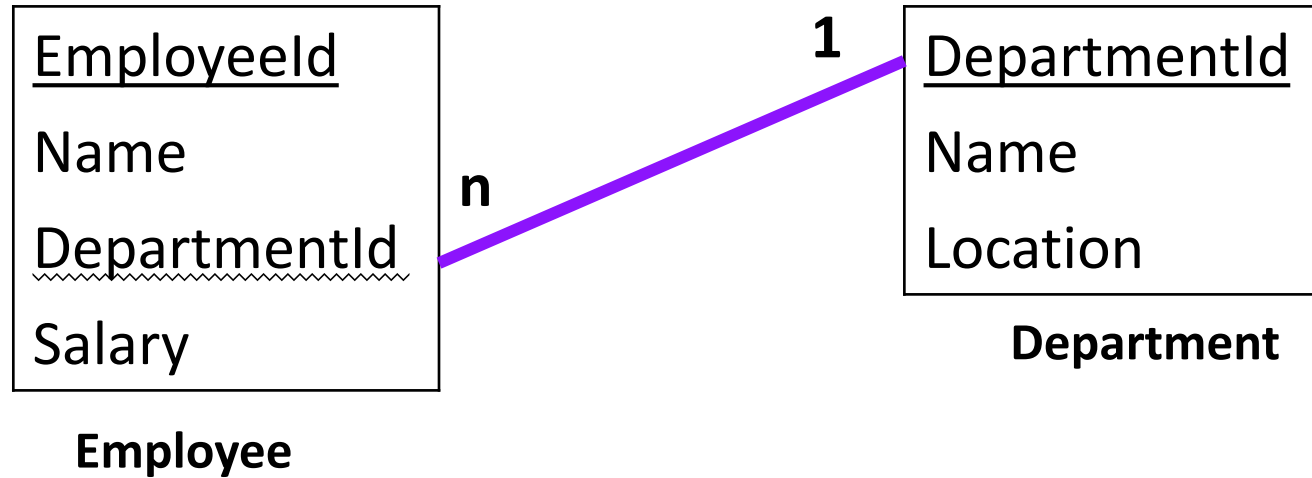
<u>employee_id</u>	employee_name	<u>department_id</u>
101	Alice	1
102	Bob	2
103	Tom	2

Employee

One-to-Many Relationship – How?

- Add a foreign key to the many side of the relationship and map it to the primary key of the related table.

One-to-Many Relationship – Example



Primary Key

<u>EmployeeId</u>	Name	<u>DepartmentId</u>
125	Nimal Silva	1
145	Ravi Fernando	3
203	Lili Perera	NULL
170	Fatima Shenaz	1

Employee

Foreign Key

n

1

Primary Key

<u>DepartmentId</u>	Name	Location
1	HR	CMB
2	ADMIN	KND
3	ENG	MTR

Department

One-to-One Relationship - What?

- If one instance of A is related to one instances of B, and one instance of B is related to one instance of A, then it is said to be a one-to-one relationship.

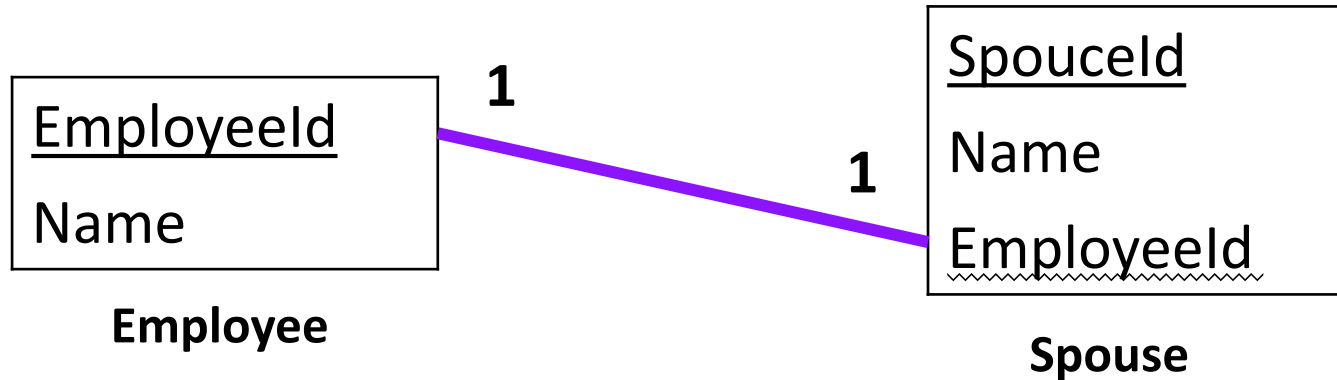
One-to-One Relationship – How?

- Approach 1
 - Add a foreign key to the one side of the relationship and make it a unique field. Then, map it to the primary key of the related table.

Note

In a weak relationship, the foreign key is added to the weak entity.

Binary One-to-One Relationship – Example



Primary Key

<u>EmployeeId</u>	Name
1	Nimal Silva
2	Ravi Fernando
3	Lili Perera
4	Fatima Shenaz

Employee

Primary Key

Foreign Key (Unique)

<u>SpouseId</u>	Name	<u>EmployeeId</u>
1	Rehana	2
2	Sumudu	3
3	Shiraz	4

Spouse

Binary One-to-One Relationship – How?

- Approach 2
 - Make primary key of the one side as the foreign key and map it to the primary key of the other side.

Note

In a weak relationship, the foreign key is added to the weak entity.

Binary One-to-One Relationship – Example



Primary Key

<u>EmployeeId</u>	Name
1	Nimal Silva
2	Ravi Fernando
3	Lili Perera
4	Fatima Shenaz

Employee

Foreign Key
Primary Key

<u>SpouseId</u>	Name
2	Rehana
3	Sumudu
4	Shiraz

Spouse

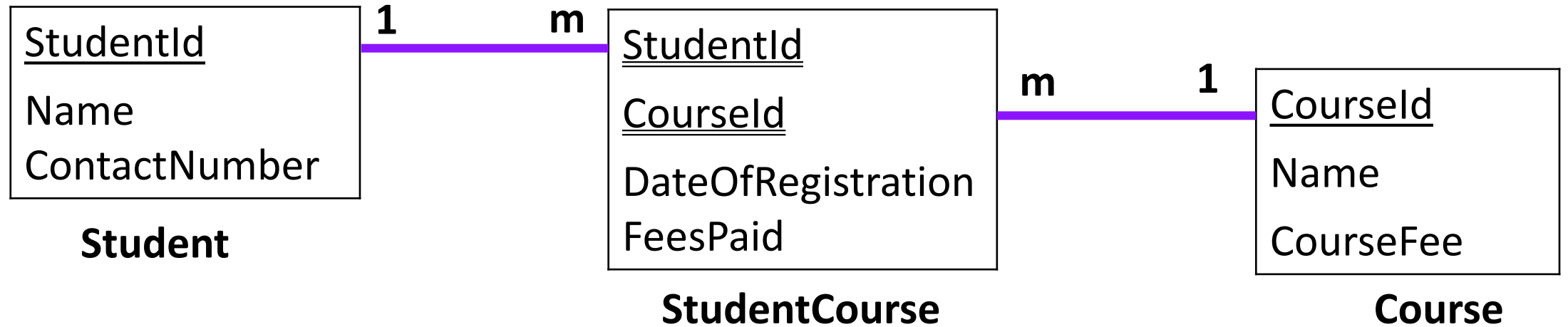
Many-to-Many Relationship - What?

- If one instance of A is related to many instances of B, and one instance of B is related to many instance of A, then it is said to be a many-to-many relationship.

Many-to-Many Relationship – How?

- Introduce a new connecting table and map the many-to-many relationship as two one-to-many relationships where many side is the connecting table.
 - Connecting table has two foreign keys forming the two one-to-many relationships.
 - The primary key of the connecting table is the combination of the two foreign keys or else we may use a new surrogated primary key.

Binary Many-to-Many Relationship – Example



Primary Key		Primary Key		
<u>StudentId</u>	Name	<u>StudentId</u>	<u>CourseId</u>	FeesPaid
1	Shehan Silva	1	1	10000
2	Ravi Fernando	1	2	35000
3	Rani Sellaiya	2	2	20000
4	Fatima Shenaz	4	3	2000

Primary Key		
<u>CourseId</u>	Name	CourseFee
1	BIT	40000
2	FIT	35000
3	AL-ICT	25000

The diagram shows the data for the Student, StudentCourse, and Course entities. The Student entity has 4 records, the StudentCourse entity has 4 records, and the Course entity has 3 records. The relationships are indicated by the 1 to m and m to 1 cardinalities.

Constraints

Constraints

- Constraints (restrictions) help enforcing validity & integrity of the data.
- Different types of constraints can be used in a database;
 - Entity integrity constraint
 - Referential integrity constraint
 - Other field level constraints

Entity Integrity Constraint

- Each record of an entity (table) has a unique identifier and that it cannot have a null value.
 - i.e Primary key value must be unique and can not be null.

Referential Integrity Constraints

- Foreign key value must be a valid value in the referencing table or it can be null.

Domain Constraints

- Domain constraints refer to the restrictions or limitations that are inherent within a particular field, subject area, or problem domain.
 - These constraints define what is permissible, valid, or feasible within that domain.
- Examples
 - Width
 - Set maximum width
 - Value ranges
 - Set valid range of values
 - Example
 - $0 \leq \text{marks} \leq 100$

Other Field Level Constraints (1)

- Unique constraint
 - Must the field values be unique?
 - Specify if duplicate value is not allowed for any two records
- NULL constraint
 - Is null allowed or not?

Data Normalization

Data Normalization – What?

- Data normalization is organizing data into relational model (relations & relationships).



Data Normalization – Why?

- Reduce the data redundancy (duplication of data)
 - Redundancy in data results in;
 - Inconsistency of data
 - Waste of storage
- Improve the integrity (relatedness) of data
 - Data records must be meaningfully related for meaningful and reliable data retrieval.

BirthCertificateId	Name	DateOfBirth
2341	Kapila Silva	1990-10-02
2342	Rehana Rafeez	2001-08-15
...	...	...

birth-certificate

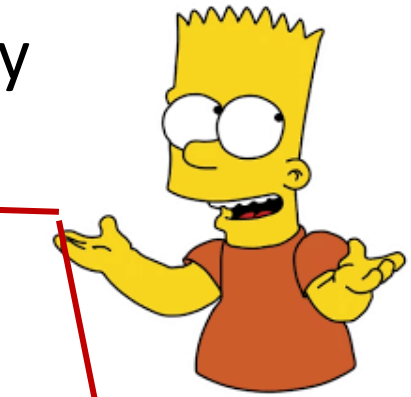
DrivingLicenceID	Name	DateOfIssue	DateOfBirth
98800	K. Silva	2020-09-18	1990-02-10
98801	R.W. Bandara	2020-09-19	1995-10-08
...	...	...	...

driving-license

Are these
related to
same person?

Inconsistency
of data?

Waste of
storage?



Data Normalization – How?

- Use data normalization during designing of relational model (i.e. Design ER diagram)
- Review and refine the relational model against data normalization theory.

Data Normalization – How?

- A sequential process
 - Complete the first step and then do the second step and so on.

1NF → 2 NF → 3NF

NF – Normal Form

Theory Behind Data Normalization

Data Normalization – The Basis

- Atomicity of data
- Functional dependency of data

Atomicity of Data

- Data field (or column or attribute) in a relation (table) must be a single unit.
 - Multiple values must not be possible.
- Examples for non atomic fields;

Example 1

Skills
Java, Python, PHP
C++, C#, Java
HTML, CSS, JavaScript

Example 2

ContactNumbers
070234567, 073456558
076889055, 011254677, 035778090
023556456, 065455955

Functional Dependency of Data – What?

- Functional dependency describes the relationship between attributes in a relation.
- Notation
 - **A** $\rightarrow$ **B** means **A** functionally determine **B**.

Functional Dependency – Example (Attributes)

IDNumber	Name	District
475848583737V	Nimal Perera	Kandy
984537373737V	Ruvini Fernando	Rathnapura
569856737878V	Kevin David	Colombo
577767444355V	Pooja Rajalingam	Jaffna
344537365437V	Ruvini Dias	Gampaha

IDNumber $\rightarrow$ Name

IDNumber $\rightarrow$ District

However;

Name $\nrightarrow$ IDNumber

Name $\nrightarrow$ District

Data Normalization Process

Introduction

- A sequential process

$1NF \rightarrow 2NF \rightarrow 3NF$

First Normal Form (1NF)

- A relation (table) is in **1NF** if all the attributes (fields) are atomic.
 - i.e. None of the fields suppose to have more than one value.
 - If such possibility is there, we need to create a new table to accommodate such multi valued field and make a one to many relationship with that table.

<u>CustomerId</u>	Name	ContactNumbers
1	Nimal Perera	011254676, 077675465
2	Kamal Gamage	071546765
3	Ravi Gunasinghe	031987865, 076543990

*Not atomic.
Violate 1NF.*

Solution

<u>CustomerId</u>	Name
1	Nimal Perera
2	Kamal Gamage
3	Ravi Gunasinghe

<u>ContactId</u>	<u>CustomerId</u>	ContactNumber
1	1	011254676
2	1	077675465
3	2	071546765
4	3	031987865
5	3	076543990

<u>ProductId</u>	ProductName	Price	Color
1	Alto	30 000 000	White,Black,Blue
2	Maruti	20 000 000	White,Black
3	Panda	25 000 000	White

*Not atomic.
Violate 1NF.*

Solution

<u>ProductId</u>	ProductName	Price
1	Alto	30 000 000
2	Maruti	20 000 000
3	Panda	25 000 000

<u>ProductColorId</u>	<u>ProductID</u>	<u>Color</u>
1	1	White
2	1	Black
3	1	Blue
4	2	White
5	2	Black
6	3	White



<u>EmployeeId</u>	Name	Qualification1	Qualification2
1	Ravi Fernando	BSc	MSc
2	Kapila Silva	BCom	
3	Chamila Gamage	CIMA	BSc

*Not atomic.
Violate 1NF.*

Solution



<u>EmployeeId</u>	Name
1	Ravi Fernando
2	Kapila Silva
3	Chamila Gamage

<u>QualificationId</u>	<u>EmployeeId</u>	Qualification
1	1	BSc
2	1	MSc
3	2	BCom
4	3	CIMA
5	3	BSc

Second Normal Form (2NF)

- What?
 - A relation is in **2NF** if it is in **1NF** AND every non-key attribute is dependent on the whole of the key (i.e. is not dependent on just part of the key);
 - i.e **no partial dependencies** for any non-key field.
 - **2NF** is more relevant when the key is composite.
- Solution
 - The field(s) which is partially dependent must be taken to the relevant table (or new table) with proper dependencies.

PK



<u>StudentId</u>	<u>CourseId</u>	Payment	CourseName
S1	C1	30000	BIT
S2	C1	20000	BIT
S2	C2	20000	BCS
S3	C2	10000	BCS
S3	C4	10000	CIMA
S4	C1	12000	BIT

$(\text{StudentId}, \text{CourseId}) \rightarrow \text{Payment}$

$\text{CourseId} \rightarrow \text{CourseName}$

CourseName violate 2NF

StudentCourse

Solution

<u>StudentId</u>	<u>CourseId</u>	Payment
S1	C1	30000
S2	C1	20000
S2	C2	20000
S3	C2	10000
S3	C4	10000
S4	C1	12000

StudentCourse

<u>CourseId</u>	CourseName
C1	BIT
C2	BCS
C4	CIMA

Course

Third Normal Form (3NF)

- What?
 - A relation is in **3NF** if it is in **2NF** AND each non-key attribute/field is directly depend on the primary key, and not indirectly depend on the primary key via another non key attribute/field.
 - i.e. **no transitive dependencies** for any non-key field.
- Solution
 - Separate the transitive dependent field(s) to another relation together with its direct dependent non-key field as the PK of the new relation.
 - The field(s) which is transitive dependent must be taken to the relevant table (or new table) with proper dependencies.

<u>BookID</u>	BookName	PublisherId	ContactNumber
B1	Gamperaliya	P1	011257867
B2	Aba Yahaluwo	P2	011256454
B3	Kaliyugaya	P1	011257867
B4	Madol Doova	P1	011257867
B5	Yashorawaya	P3	011267543

BookId → BookName

BookId → PublisherId

PublisherId → ContactNumber

ContactNumber violate 3NF

Book

Solution

<u>BookID</u>	BookName	<u>PublisherId</u>
B1	Gamperaliya	P1
B2	Aba Yahaluwo	P2
B3	Kaliyugaya	P1
B4	Madol Doova	P1
B5	Yashorawaya	P3

Book

<u>PublisherID</u>	Publisher	ContactNumber
P1	Sarasavi	011257867
P2	Godage	011256454
P3	Thisara	011267543

Publisher

SQL

Introduction

- SQL stands for **Structured Query Language**.
- SQL is an ANSI standard supported by major relational databases.
- SQL is a 4GL.
 - In SQL we code 'what to do?' rather than 'how to do?'.
- When a SQL query is sent to a relational DBMS, it is executed in the DBMS and results are sent back.

How to Use SQL?

- A developer execute SQL using;
 - Command line
 - Other database tools
- A program sends embedded SQL query to the database and get the results.

```
C:\Windows\System32\cmd.exe - mysql -u root -p
mysql> update user set city='Kanpur' where id=2;
Query OK, 1 row affected (0.09 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from user;
+----+-----+-----+-----+
| id | name  | city  | age  |
+----+-----+-----+-----+
| 1  | Naman | Delhi | 28   |
| 2  | Naman | Kanpur | 28   |
| 3  | Vinod | Lucknow | 26   |
| 4  | Rajat | Unnao  | 30   |
| 5  | Shailja | Fatehpur | 24   |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

```
8 $conn = new mysqli("localhost", "root", "", "sem3");
9 if ($conn->connect_error) {
10     die("Connection failed" . $conn->connect_error . "<br/>");
11 }
12
13 $sql = "UPDATE employee SET name = '$name', gender = '$gender',
14       salary = $salary, dob = '$dob'
15       WHERE id = $id";
16
17 if ($conn->query($sql) === TRUE) {
18     echo "Employee updated successfully";
19     header('Location: list.php');
20 } else {
21     echo "Error updating employee: ".$sql."<br>".$conn->error;
22 }
```

SQL Statements - Classification

- Data Definition Language (DDL)
 - **CREATE**
 - **ALTER**
 - **DROP**
- Data Manipulation Language (DML)
 - **INSERT**
 - **DELETE**
 - **UPDATE**
- Data Query Language (DQL)
 - **SELECT**

Data Definition Language (DDL)

Overview

- DDL statements let a user create and change the structure (relations and relationships).
 - **CREATE**
 - **DROP**
 - **ALTER**

CREATE Statements

- Use to create
 - Table (Relation)
 - Fields
 - Primary key
 - Other field level constraints
 - Relationship
 - Foreign key
 - Referential integrity constraints
 - Index on field(s)

CREATE TABLE - Syntax

```
CREATE TABLE <table name> (  
    <field name> <data type> <constraints>,  
    <field name> <data type> <constraints>,  
    ..  
    <Other constraints>  
)
```

CREATE TABLE - Example

```
CREATE TABLE department (  
    id            int,  
    name          varchar(15),  
    location      varchar(15)  
)
```

ANSI/ISO SQL Data Types

Data type	Description
INT / INTEGER	Integer numbers
NUMBER / NUMERIC / DECIMAL / DEC / FLOAT / REAL	Floating point decimal numbers
DOUBLE	Double precision decimal numbers
CHAR / CHARACTER	Fixed length character
VARCHAR	Variable-length character strings
TEXT	Long text
MONEY / CURRENCY	Money amount
DATE / TIMESTAMP	Dates and times
BOOLEAN / LOGICAL	Boolean data

Primary Key – Approach 1

```
CREATE TABLE department (  
    department_id    INT NOT NULL PRIMARY KEY,  
    name             VARCHAR(15),  
    location         VARCHAR(15)  
)
```

Notice – Can not be used for composite primary key.

Primary Key – Approach 2

```
CREATE TABLE department (  
    department_id      INT NOT NULL,  
    name               VARCHAR(45) ,  
    location           VARCHAR(45) ,  
    PRIMARY KEY (department_id)  
)
```

Single field primary key.

```
CREATE TABLE employee_project (  
    employee_id        INT NOT NULL ,  
    project_id         INT NOT NULL ,  
    budget             DECIMAL,  
    PRIMARY KEY (employee_id, project_id)  
)
```

Multi-field primary key.

Primary Key – Approach 3

```
CREATE TABLE department (  
    department_id INT NOT NULL,  
    name          VARCHAR(45),  
    location      VARCHAR(45),  
    CONSTRAINT pk_dep PRIMARY KEY(department_id)  
)
```

Single field primary key.

```
CREATE TABLE employee_project (  
    employee_id INT NOT NULL ,  
    project_id  INT NOT NULL ,  
    budget      DECIMAL,  
    CONSTRAINT pk_emp_proj PRIMARY KEY (employee_id, project_id)  
)
```

Multi-field primary key.

Foreign Key – Approach 1

```
CREATE TABLE employee (  
    employee_id      INT NOT NULL ,  
    name             VARCHAR(45) ,  
    salary            DECIMAL ,  
    department_id    INT ,  
    PRIMARY KEY (id) ,  
    FOREIGN KEY (department_id) REFERENCES department (department_id)  
)
```

Foreign Key – Approach 2

```
CREATE TABLE employee (  
    employee_id      INT NOT NULL ,  
    name             VARCHAR(45) ,  
    salary            DECIMAL ,  
    department_id     INT ,  
    CONSTRAINT pk_emp PRIMARY KEY (employee_id) ,  
    CONSTRAINT fk_em_dep FOREIGN KEY (department_id)  
                                REFERENCES department(department_id)  
)
```

Referential Integrity Constraints

- ON UPDATE

RESTRICT | CASCADE | SET NULL |
NO ACTION | SET DEFAULT

- ON DELETE

RESTRICT | CASCADE | SET NULL |
NO ACTION | SET DEFAULT

Referential Integrity Constraints

- **CASCADE**

- When a parent record is deleted/updated, update/delete the related rows in the child table.

- **RESTRICT**

- Deleting/Updating a record from the parent table is not allowed, if there are related rows in the child table.

- **SET NULL**

- When a parent record is deleted/updated, set the value of the foreign key field of related rows in the child table to the **null** value.

- **SET DEFAULT**

- When a parent record is deleted/updated, sets the value of the foreign key field of related rows in the child table to the default value of that field.

Referential Integrity Constraints – Example 1

```
CREATE TABLE employee (
    employee_id      INT NOT NULL ,
    name             VARCHAR(45) ,
    salary            DECIMAL,
    department_id     INT,
    PRIMARY KEY (id),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
                                ON DELETE CASCADE
                                ON UPDATE RESTRICT
)
```

Referential Integrity Constraints – Example 2

```
CREATE TABLE employee (  
    employee_id      INT NOT NULL ,  
    name             VARCHAR(45) ,  
    salary            DECIMAL ,  
    department_id    INT ,  
    CONSTRAINT pk_emp PRIMARY KEY (employee_id) ,  
    CONSTRAINT fk_em_dep FOREIGN KEY (department_id)  
        REFERENCES department(department_id)  
        ON DELETE CASCADE  
        ON UPDATE RESTRICT  
)
```

Referential Integrity Constraints – Exercise

emp_id	name	salary	dep_id
1	Nimal	80000	1
2	Kamal	90000	1
3	Ravi	70000	2
4	Dilini	70000	3
5	Ruvini	85000	NULL

employee

RESTRICT | CASCADE | SET NULL |
NO ACTION | SET DEFAULT

dep_id	dep_name	location
1	HR	Colombo
2	FIN	Kandy
3	ADMIN	Galle
4	MKT	Gampaha

department

What happen when;
delete this record?
update the dep_id of this record?

Other Field Level Constraints

- **NOT NULL**
 - Null value is not allowed.
- **UNIQUE**
 - Value can not be duplicated.
- **DEFAULT**
 - Specify the default value if no value is specified.
- **CHECK**
 - Specify any other validations combined by logic operators. (Not supported by Mysql, instead need to use triggers)

```
CREATE TABLE employee (  
    employee_id    CHAR(5)    NOT NULL,  
    NIC            CHAR(10)   UNIQUE,  
    address        VARCHAR(50),  
    salary         DEC(7,2)   CHECK (Salary >= 0),  
    gender         CHAR(1)    CHECK (Gender IN ('M', 'F')),  
    DOB           DATE,  
    first_name     CHAR(10),  
    mid_initials   CHAR(10),  
    last_name      CHAR(15)   NOT NULL,  
    dept_No       CHAR(2)    NOT NULL DEFAULT 10,  
    supervisor     CHAR(5),  
    PRIMARY KEY (employee_id),  
    FOREIGN KEY (dept_no) REFERENCES department(department_id),  
    FOREIGN KEY (supervisor) REFERENCES employee(employee_id)  
)
```

Specify Compound Constraints - Example

```
CREATE TABLE employee (  
    employee_id      INT NOT NULL PRIMARY KEY,  
    name             VARCHAR(45) ,  
    salary            DECIMAL ,  
    CONSTRAINT chk_emp CHECK (employee_id > 0 AND salary > 0)  
)
```

DROP Statements

- Use to drop table, view (cover later) or index
- Syntax

```
DROP TABLE <table-name>
```

- Examples

```
DROP TABLE department;
```

ALTER Table Statements

- Use to alter the structure or add/remove constraints of a table.

ALTER Table - Examples

Rename table

```
ALTER TABLE billing  
RENAME TO order;
```

Adding a column

```
ALTER TABLE employee  
ADD dob DATE;
```

Drop a column

```
ALTER TABLE department  
DROP COLUMN dob ;
```

Rename a column

```
ALTER TABLE department  
CHANGE COLUMN name  
dep_name VARCHAR(20) ;
```

Data Manipulation Language (DML)

Data Manipulation Language (DML) - Statements

- SQL DML statements are;
 - **INSERT**
 - **UPDATE**
 - **DELETE**

INSERT Statement

- Use to insert one record or set of records to a table or view.
- Can be used;
 - With column names
 - Preferred approach
 - Without column names
 - Relies on column order of the table

INSERT – Approach 1 (Specify Field Names)

Syntax

```
INSERT INTO table_name (column1, column2, column3,...)  
VALUES (value1, value2, value3,...),  
        (value1, value2, value3,...),...;
```

Examples

```
INSERT INTO department (department_id, name, location)  
  VALUES (5, 'ADMIN', 'Rathnapura');
```

```
INSERT INTO department (department_id, name, location)  
  VALUES (6, 'PROD', 'Matara'), (7, 'PKG', 'Kalutara');
```

Note - Order of columns is not necessarily the same as in the table, but the order of values must be same as the order of specified columns in the statement.

INSERT – Approach 2 (Not Specify Field Names)

Syntax

```
INSERT INTO table_name  
VALUES (value1, value2, value3,...),  
        (value1, value2, value3,...), ..;
```

Examples

```
INSERT INTO department VALUES (1, 'ENG', 'kandy');
```

```
INSERT INTO department VALUES (2, 'MKT', 'Jaffna'), (3, 'ADM', 'Galle');
```

Note

- Values must be in the same order as the fields in the table.
- You cannot skip any of the columns (Use **null** if value is not specified).

UPDATE - Statement

- Use to update data of one or more records of a table.

Syntax

```
UPDATE table_name  
SET column1=value,  
      column2=value2, ...  
[WHERE Condition]
```

Note – If the WHERE condition is not used, all the records in that table is updated.

Example (Update all the records)

```
UPDATE department  
SET location = 'London'
```

Example (Update selected records)

```
UPDATE department  
SET location = 'colombo'  
WHERE department_id = 2
```

Example (Update selected records with a subquery)

```
UPDATE employee  
SET salary = 30000  
WHERE department_id IN (SELECT id FROM  
department WHERE budget > 20000)
```

DELETE Statement

- Use to delete all or some records in a table.

Syntax

```
DELETE FROM table_name  
WHERE <condition>
```

Note – If the WHERE condition is not used, all the records in that table is deleted.

Example (Delete all the records)

```
DELETE FROM employee
```

Example (Delete selected records)

```
DELETE FROM employee  
WHERE employee_id = 3
```

Example (Delete selected records with a subquery)

```
DELETE FROM employee  
WHERE employee_id =  
    (SELECT employee_id FROM employee  
        WHERE department_id = 1)
```

Data Query Language (DQL)

employee_id	name	salary	department_id
1	Kevin	20000	3
2	KAMAL	60000	1
3	Rehana	80000	3
4	raju	70000	2
5	Ganesh	80000	NULL

employee

department_id	name	location	budget
1	HR	kandy	1500000
2	FIN	Vauniya	2000000
3	MKT	gampaha	NULL
4	PROD	Galle	2500000

department

employee_id	project_id	hours_worked
1	1	10
1	2	5
3	3	10
4	1	15
4	3	10
5	2	10

Employee_project

project_id	name
1	PR1
2	PR2
3	PR3

project

SELECT - Data Query Language

- Use to retrieves data from one or more tables.

Syntax

```
SELECT <[DISTINCT] * or select-item,..>  
FROM <table1,..>  
[WHERE <search conditions>]  
[GROUP BY <group-columns,>]  
[HAVING <search conditions>]  
[ORDER BY <ASC | DESC>]
```

Example (Retrieve all fields of all the records of a table)

```
SELECT *  
FROM employee
```

Example (Retrieve some fields of all the records of a table)

```
SELECT employee_id, name, salary  
FROM employee
```

Example (Retrieve all fields of all the records of a table)

```
SELECT *  
FROM employee  
WHERE department_id = 2
```

Example (Retrieve some fields of some of the records of a table)

```
SELECT employee_id, name, salary  
FROM employee  
WHERE department_id = 2
```

Provide Alias for Retrieved Fields

- By default, the retrieved field names are same as column names in the table.
- We may provide a different name for the fields using **AS** key word.

```
SELECT employee_id AS eid, name AS ename  
FROM employee  
WHERE department_id = 3;
```

+-----+-----+		
eid	ename	
+-----+-----+		
1	Kevin	
3	Rehana	
+-----+-----+		

Use of **DISTINCT**

- If the query results in duplicate rows, we can use **DISTINCT** to eliminate duplicates.

```
SELECT department_id FROM employee
```

department_id
NULL
1
2
3
3

```
SELECT DISTINCT department_id FROM employee
```

department_id
NULL
1
2
3

Sort the Retrieved Records

- By default, the SELECT query, retrieve and present data in the order records are in the table.
- We can sort the resulting records by any field using **ORDER BY ASC|DESC**
- **ASC** is the default value for **ORDER BY**.

```
SELECT name, salary
FROM employee
ORDER BY name ASC;
```

name	salary
Ganesh	80000
KAMAL	60000
Kevin	20000
raju	70000
Rehana	80000

```
SELECT name, salary
FROM employee
ORDER BY name DESC;
```

name	salary
Rehana	80000
raju	70000
Kevin	20000
KAMAL	60000
Ganesh	80000

SQL - Operators

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to
BETWEEN	Between an inclusive range
IS NULL	Check if value is null
IN (v1,v2,...)	Is the value in the list

```
SELECT *  
FROM employee  
WHERE name = 'vikum';
```

```
SELECT *  
FROM employee  
WHERE salary >= 200000;
```

```
SELECT *  
FROM employee  
WHERE department_id <> 2;
```

```
SELECT *  
FROM employee  
WHERE salary BETWEEN 100000 AND 150000;
```

```
SELECT *  
FROM employee  
WHERE department_id IS NULL;
```

```
SELECT *  
FROM employee  
WHERE department_id IS NOT NULL;
```

```
SELECT *  
FROM employee  
WHERE department_id IN (1, 3);
```

```
SELECT *  
FROM employee  
WHERE department_id NOT IN (1, 3);
```

LIKE - Operator

- Use for pattern matching.
 - `_` represent a single character.
 - `%` represent one or many characters.
 - Pattern characters are not case sensitive.
- Examples
 - `'%L%'` - Values that contains the letter 'L' or 'l'
 - `'La%'` - Values that start with starts with 'LA', 'la', 'La' or 'lA'
 - `'b%s'` - Values that start with 'b' or 'B' and ends with 's' or 'S'
 - `'_r__%'` - 4 letter values with 'r' or 'R' at second letter
 - `'_r%'` - Values having 'r' or 'R' in the second position
 - `'Ravi'` - Values having values like 'RAVI', 'ravi', 'Ravi', 'rAvI', etc..

```
SELECT *  
FROM employee  
WHERE name LIKE 'ravi';
```

```
SELECT *  
FROM employee  
WHERE name LIKE '%L%';
```

```
SELECT *  
FROM employee  
WHERE name LIKE '_r__%';
```

Combining SELECT Conditions

- SELECT conditions can be combined using following operators;

NOT

AND

OR



Operator
precedence

- Brackets can be used to override the precedence.

```
SELECT *  
FROM employee  
WHERE salary >= 100000 AND salary <= 150000  
      OR department_id = 3;
```

The above query is same as the following.

```
SELECT *  
FROM employee  
WHERE (salary >= 100000 AND salary <= 150000)  
      OR department_id = 3;
```

Joins

- Use to connect or relate tables to get combine results.

Inner Join – What?

- Get only common records based on joining field.

Inner Join – How ?

Approach 1

```
SELECT column(s)  
FROM table1  
INNER JOIN table2  
ON table1.column1 = table2.column2
```

```

SELECT *
FROM employee
INNER JOIN department
ON employee.department_id = department.department_id;

```

employee_id	name	salary	department_id	department_id	name	location	budget
2	KAMAL	60000	1	1	HR	kandy	1500000
4	raju	70000	2	2	FIN	Vauniya	2000000
1	Kevin	20000	3	3	MKT	gampaha	NULL
3	Rehana	80000	3	3	MKT	gampaha	NULL

Inner Join – How ?

Approach 2

```
SELECT column(s)  
FROM table1, table2  
WHERE table1.column1 = table2.column2
```

```

SELECT *
FROM employee, department
WHERE employee.department_id = department.department_id;

```

employee_id	name	salary	department_id	department_id	name	location	budget
2	KAMAL	60000	1	1	HR	kandy	1500000
4	raju	70000	2	2	FIN	Vauniya	2000000
1	Kevin	20000	3	3	MKT	gampaha	NULL
3	Rehana	80000	3	3	MKT	gampaha	NULL

Thank You

Aurora Computer Studies | auroracs.lk

